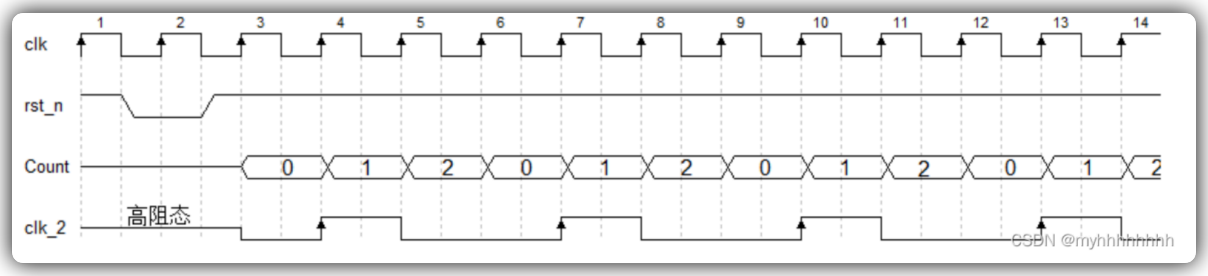
module 01: 奇数分频

1. 奇数分频

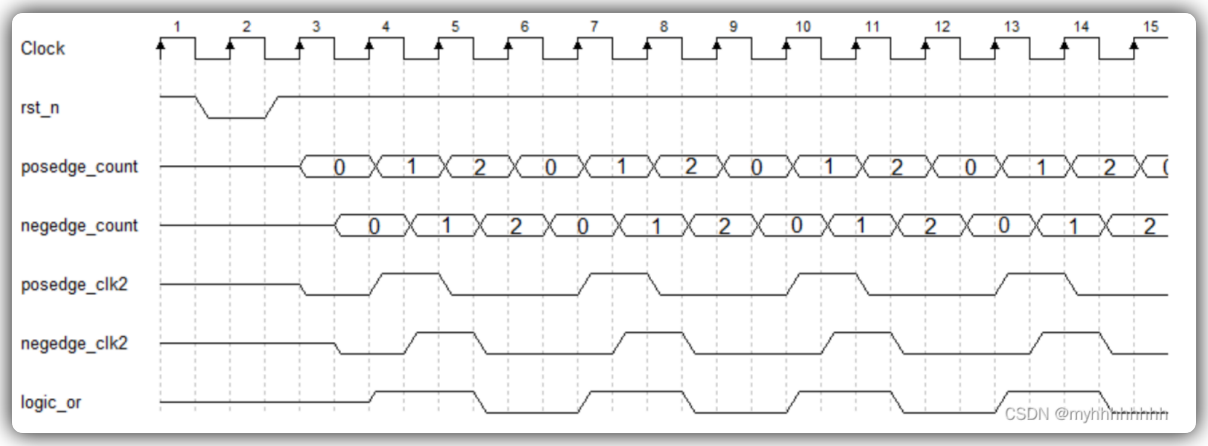
- 实现三分频电路，不需要满足50%占空比。
- 实现五分频电路，不需要满足50%占空比。
- 实现任意奇数分频，不需要满足50%占空比。
- 实现奇数分频，需要满足占空比50%。

2. 奇数分频电路原理

- 不需要满足50%占空比的分频电路
  - 假如Clock是输入的时钟信号，Clock2是输出的非50%占空比的三分频信号，那么对于计数器，Clock2实现的操作是：当采样边沿每次遇到count=0的时候进行翻转，每次遇到1的时候再次完成翻转



- 需要满足50%占空比的分频电路
  - 假如Clock是输入的时钟信号，logic_or是输出的50%占空比的三分频信号，那么我们需要两个计数器，一个上升沿计数器，一个下降沿计数器，计数的最大值都是2，对待这两个计数器，我们可以执行相同的操作：**即在边沿采样的时候如果发现计数器的值是1或者2，那么则翻转信号clk2，体现在posedge_clk2和negedge_clk2上，这两个信号取”相或“操作，得到logic_or就是50%占空比的分频电路**



3. RTL:非50%占空比的三分频电路

- 这里只需要修改N的值，即可改变分频，如N=5，同时改变count的位宽，即可实现五分频，以此类推，即可实现不要求占空比的任意分频电路.

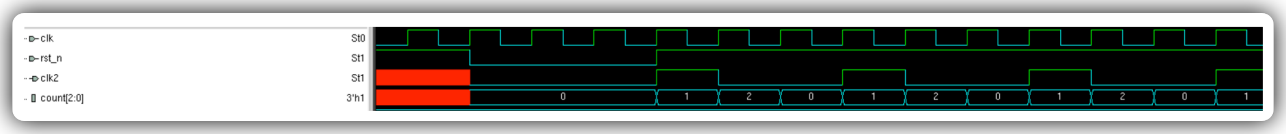
```
module f3(clk, rst_n, clk2);
   input  clk;
   input  rst_n;
   output clk2;

   reg      clk2_r;
   assign    clk2 = clk2_r;

   reg [2:0] count;
   parameter N = 3;

   always@(posedge clk or negedge rst_n)
   begin
    if(!rst_n || count == N-1)
       count <= 3'd0;
    else
       count <= count +1'd1;
   end

   always@(posedge clk or negedge rst_n)
   begin
    if(!rst_n)
       clk2_r <= 3'd0;
    else if (count == 3'd0 || count == 3'd1)
       clk2_r <= !clk2_r;
    else
       clk2_r <= clk2_r;
   end
endmodule
```



4. RTL:50%占空比的奇数分频电路

- 若想将其扩展到7分频，9分频，只需要修改N和count的位数即可.

```
module f3(clk, rst_n,clk2);
   input clk;
   input rst_n;
   output clk2;

   reg [2:0] negedge_count;
   reg [2:0] posedge_count;
   reg posedge_clk2;
   reg negedge_clk2;
   parameter N = 3;

   always@(posedge clk or negedge rst_n)
   begin
    if(!rst_n || posedge_count == N-1)
       posedge_count <= 3'd0;
    else
       posedge_count <= posedge_count +1'd1;
   end

   always@(negedge clk or negedge rst_n)
   begin
    if(!rst_n || negedge_count == N-1)
       negedge_count <= 3'd0;
    else
       negedge_count <= negedge_count +1'd1;
   end

   always@(posedge clk or negedge rst_n)
   begin
    if(!rst_n)
       posedge_clk2 <= 1'd0;
    else if (posedge_count == 3'd1 || posedge_count == 3'd0)
       posedge_clk2 <= !posedge_clk2;
    else
       posedge_clk2 <= posedge_clk2;
   end

   always@(negedge clk or negedge rst_n)
   begin
    if(!rst_n)
       negedge_clk2 <= 1'd0;
    else if (negedge_count == 3'd1 || negedge_count == 3'd0)
       negedge_clk2 <= !negedge_clk2;
    else
       negedge_clk2 <= negedge_clk2;
   end

   assign clk2 = posedge_clk2 || negedge_clk2;

endmodule
```

